

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot B

COURSE OUTCOME COVERED

CO3: Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration :60 Minutes

Annexure A

Case Study

Code of Conduct:

I S.Jeswanth Kumar (192321056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Jeswanth Kumar

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent. Analyze the issue and explain how preprocessing improves feature detection performance.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching. Analyze why edge detection is insufficient and justify the use of corner detection in this case.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results. Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy. Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

Answers:

1. Issue Analysis:

When images are captured under varying lighting conditions (shadows, overexposure, low light, uneven illumination), the raw pixel intensity values become inconsistent across images of the same scene.

Feature detectors (like corner or edge detectors) rely on local intensity gradients to identify keypoints.

Under poor or variable lighting:

- Gradients get distorted — weak edges may vanish, and noise-induced gradients may create false features.
- The same physical feature can produce different intensity/gradient responses in different images, breaking repeatability.
- Detection thresholds calibrated for one lighting condition fail under another, causing missed detections or false positives.

How Preprocessing Improves Performance:

- **Grayscale conversion & normalization:** Removes color-channel inconsistency and standardizes intensity range (0–255 → normalized scale).
- **Histogram Equalization** : Redistributes intensity values to enhance contrast in poorly lit regions without over-amplifying noise.
- **Gaussian smoothing:** Reduces high-frequency noise that could be mistaken for features, while preserving genuine structural edges.
- **Illumination normalization** (e.g., homomorphic filtering): Separates illumination component from reflectance component so detection depends more on object structure than lighting.
- **Gamma correction:** Adjusts non-linear intensity response of the sensor to standardize brightness perception.

Conclusion: Preprocessing makes gradient responses more stable and repeatable across varying capture conditions, which directly improves the reliability and consistency of downstream feature detection.

2. Why Edge Detection Is Insufficient:

Edge detectors (e.g., Sobel, Canny) identify pixels where intensity changes sharply in one direction along the gradient normal. However:

- Edges only mark boundaries, not distinct, localizable points.
- Along a straight edge, every point looks locally identical (this is called the aperture problem) —there's no way to uniquely match one point on the edge with the corresponding point in another image, since displacement along the edge direction is ambiguous.
- Edges in textured regions become dense and fragmented, making it hard to establish reliable one-to-one correspondences for matching.

Why Corner Detection Is Justified:

A corner is a point where intensity changes significantly in two orthogonal directions simultaneously (high gradient variance in all directions). This gives corners key properties edges lack:

- **Uniqueness/Locality:** A corner has a distinctive local neighborhood, unlike a point on a straight edge.
- **Well-defined position:** Unlike edges, corners can be pinpointed precisely (no aperture-problem ambiguity).
- **Repeatability:** Detectors like Harris Corner Detector or Shi-Tomasi use the second-moment matrix (structure tensor) of local gradients; large eigenvalues in both directions indicate a strong, matchable corner.
- **Better for matching/tracking:** Since corners are well-localized in 2D, they can be reliably matched between images, which is essential for tasks like stereo correspondence, tracking, and homography estimation.

Conclusion: In textured regions with well-defined structure, corner detection succeeds where edge detection fails because it provides uniquely identifiable, spatially-precise keypoints suitable for matching.

3. Effect of Noise on Detection:

The Hough Transform detects lines by mapping edge points into a parameter space (ρ , θ) and finding peaks where many points vote for the same line. Noise degrades this process because:

- **Spurious edge points** from noise generate false votes scattered across the accumulator (parameter) space, creating misleading local peaks.
- **Weak/broken edges:** Noise can suppress true edge pixels (via inconsistent gradients), causing gaps in a real line, which splits votes across multiple nearby (ρ , θ) bins instead of concentrating them into a single strong peak.
- **Peak ambiguity:** Multiple close, weak peaks appear instead of one dominant peak, making it hard to distinguish true lines from noise-induced clusters.
- **False positives:** Random noise alignments can occasionally accumulate enough votes to be misidentified as genuine lines.

Suggested Improvements:

Preprocessing-based:

- Apply Gaussian blurring before edge detection to suppress high-frequency noise.
- Use Canny edge detector (instead of simple gradient thresholding) since it includes non-maximum suppression and hysteresis thresholding, producing cleaner, thinner edges.
- Apply median filtering for salt-and-pepper type noise.

Parameter-based:

- Increase the accumulator threshold (minimum votes needed to accept a peak) to reject weak, noise-driven peaks.
- Tune ρ and θ resolution (bin size) — coarser bins reduce sensitivity to small noisy deviations, though at some cost to precision.
- Use Probabilistic Hough Transform (PHT), which works on a subset of edge points and also returns line segment endpoints, making it more robust and computationally efficient in noisy conditions.

- Apply minimum line length and maximum line gap parameters (as in PHT) to filter out spurious short segments caused by noise.

Conclusion: Combining noise-reduction preprocessing with careful parameter tuning (and using variants like PHT) significantly improves the robustness and accuracy of line detection in noisy images.

4. Why Traditional Matching Fails:

Traditional feature matching techniques often rely on fixed-scale, orientation-sensitive descriptors or simple correlation/template matching. When two images of the same object are captured at different scales and orientations:

- A fixed-size detection window cannot capture the same real-world structure consistently — a feature large in one image may appear small in another.
- Gradient/descriptor patterns are orientation-dependent, so rotated versions of the same feature produce different descriptor values, breaking correspondence.
- This leads to poor keypoint repeatability and incorrect matches.

Why Scale-Invariant Methods Work Better:

Algorithms like SIFT (Scale-Invariant Feature Transform) and SURF (Speeded-Up Robust Features) are explicitly designed to handle scale and rotation changes:

- **Scale-space extrema detection:** Features are detected across a pyramid of images at multiple scales (using Difference of Gaussians), so the same physical feature is detected consistently regardless of the image's scale.
- **Characteristic scale assignment:** Each keypoint is assigned a scale at which it was detected, allowing descriptors to be computed relative to that scale — making comparisons scale-invariant.
- **Orientation assignment:** Each keypoint is assigned a dominant orientation based on local gradient directions; the descriptor is then computed relative to this orientation, making it rotation-invariant.
- **Robust, high-dimensional descriptors:** SIFT's 128-dimensional descriptor captures the local gradient distribution in a way that remains stable under affine changes, illumination variation, and moderate viewpoint changes.

Conclusion: Because scale-invariant methods normalize for scale and orientation before descriptor computation, the same physical feature yields nearly identical descriptors in both images despite the geometric transformation — enabling accurate matching where traditional fixed-scale methods fail.

5. Problem: High Dimensionality and Redundancy

When a system extracts a large number of features (e.g., high-dimensional descriptors like SIFT vectors, or raw pixel-based features), it faces:

- **High computational cost:** Matching, storage, and processing scale poorly with dimensionality.
- **Redundancy:** Many features are highly correlated and carry overlapping information, adding little discriminative value.
- **Curse of dimensionality:** In very high-dimensional spaces, distance-based matching becomes less meaningful, and classifiers may overfit.

How PCA Helps:

Principal Component Analysis (PCA) is a linear dimensionality reduction technique that works as follows:

1. Computes the covariance matrix of the feature vectors to capture correlations among dimensions.
2. Finds the eigenvectors (principal components) of this covariance matrix — these represent directions of maximum variance in the data.
3. Ranks components by their eigenvalues (amount of variance explained).
4. Projects the original high-dimensional features onto the top-k principal components ($k \ll$ original dimension), retaining most of the meaningful variance while discarding redundant/low-variance dimensions.

Benefits:

- **Dimensionality reduction:** Reduces feature vector size significantly (e.g., 128-D SIFT descriptors reduced to 20–40 dimensions) while preserving most discriminative information.
- **Removes redundancy:** Since principal components are orthogonal (uncorrelated), redundant correlated features are eliminated.
- **Improves computational efficiency:** Faster matching, less memory, quicker training for downstream classifiers.
- **Reduces noise:** Low-variance components often correspond to noise; discarding them can improve robustness.
- **Mitigates overfitting:** Especially useful when the number of features is large relative to the number of training samples.

Conclusion: PCA optimizes the feature set by transforming correlated, high-dimensional features into a compact set of uncorrelated components that retain maximum variance, thereby reducing computational cost and redundancy while preserving representational quality.